



Manuel Utilisateur

Projet Genie Logiciel - gl16

ENSIMAG 2025/2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

1 Description du compilateur du point de vue de l'utilisateur

Le compilateur a pour objectif de traduire un programme écrit en Deca vers une forme exécutable, tout en garantissant au maximum la correction du programme dès la compilation. Dans le cadre du projet GL, Deca sert de support pour mettre en œuvre les étapes d'un compilateur, telles que analyse lexicale, analyse syntaxique et vérification contextuelle.

L'utilisateur attend d'un compilateur qu'il impose un cadre strict, un programme qui ne respecte pas les règles du langage doit être rejeté de manière explicite, avec un diagnostic compréhensible et localisé. Cette exigence est d'autant plus importante que Deca est destiné à être compilé vers un assembleur de machine abstraite : une erreur détectée trop tard peut se transformer en comportement imprévisible à l'exécution. Le compilateur agit donc comme un filtre, dont le rôle est de garantir que la sortie générée correspond à une interprétation cohérente du langage.

Le programme `decac` est l'outil fourni à l'utilisateur pour réaliser cette compilation. Il prend en entrée un fichier `.deca`, applique successivement les phases d'analyse et produit une sortie exploitable. En cas d'erreur, l'un des objectifs de `decac` est d'arrêter la compilation en fournissant un diagnostic avec nature de l'erreur et position dans le fichier. Le compilateur vise à supporter à la fois le sous-ensemble sans objet et l'extension avec objet, de manière cohérente avec les règles du langage et les fonctionnalités implémentées.

Du point de vue de l'utilisation, la commande `decac` doit s'intégrer dans un processus habituel : modification du fichier source, compilation, puis exécution du code produit. On attend un comportement stable : à programme et options identiques, le compilateur produit la même sortie et signale les mêmes erreurs. C'est important, pour le débogage et la validation automatique via des tests. Cette stabilité suppose aussi que les diagnostics soient constants et que la compilation n'interagisse pas de manière inattendue avec l'entrée standard.

Le compilateur cible la machine abstraite IMA, conçue pour exécuter des programmes Deca. Si la compilation réussit, un fichier `.ass` est généré dans le même répertoire que le fichier source. Ce fichier contient le code assembleur destiné à être exécuté par l'outil IMA.

Du point de vue de l'utilisateur, le compilateur doit rester prévisible : à code et options identiques, le résultat est identique. Il doit également produire des messages qui indiquent la zone du code concernée et la règle violée.

Le processus interne du compilateur est organisé en trois étapes :

Étape A (analyse lexicale et analyse syntaxique) : le compilateur lit le texte et le transforme en unités lexicales (mots-clés, identificateurs, littéraux, symboles), puis construit un arbre de syntaxe abstraite (AST). Il vérifie que la séquence de tokens respecte la grammaire du langage Deca. Si la structure attendue n'est pas respectée (parenthèses manquantes, mot-clé absent, point-virgule oublié, etc.), une erreur de syntaxe est signalée.

Étape B (vérifications contextuelles) : le compilateur vérifie la validité sémantique du programme (typage, portée, cohérence) et enrichit l'AST avec des informations contextuelles. Cette phase garantit qu'un programme syntaxiquement correct respecte aussi les règles du langage.

Étape C (génération de code) : le compilateur traduit l'arbre décoré en instructions

pour la machine abstraite IMA. Le fichier produit peut ensuite être exécuté avec l'outil prévu dans le projet.

Comme illustré à la Figure 1, le pipeline s'enchaîne de manière séquentielle, chaque étape dépend de la précédente. Pour l'utilisateur, cela se traduit par une progression simple : une erreur lexicale bloque la suite de l'analyse ; une erreur syntaxique empêche les vérifications contextuelles ; une erreur contextuelle stoppe la génération de code afin d'éviter un assembleur incohérent. Cela explique pourquoi certains diagnostics apparaissent tôt : la compilation s'arrête au premier niveau de violation détectable.

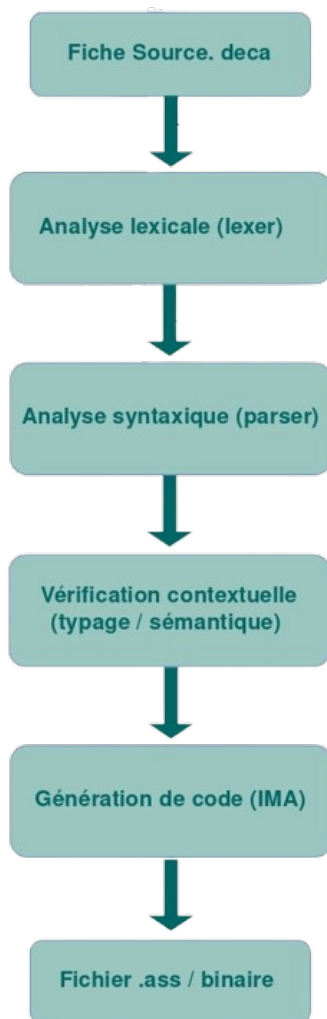


FIGURE 1 – Schème du compilateur

1.1 Les sous-ensembles du langage, sans objet

Ce sous-ensemble correspond à la base du langage et constitue un premier objectif d'implémentation. Il concerne des programmes structurés avec des variables globales et des instructions dans un bloc principal.

Dans ce mode, l'utilisateur utilise un langage impératif avec déclarations, expressions arithmétiques et booléennes, structures de contrôle et procédures d'affichage. Le point

principal est la cohérence de la compilation, les erreurs doivent être détectées de manière fiable et le code généré doit rester correct, lorsque les ressources (registres) sont limitées. Le sous-ensemble sans objet sert aussi de référence pour valider l'ensemble du pipeline (analyse, vérifications, génération) avant d'introduire les mécanismes orientés objet.

En mode sans objet, plusieurs constructions de la programmation orientée objet ne sont pas prises en charge. Ces restrictions ne correspondent pas à des erreurs, mais à une définition du sous-ensemble. Elles impliquent que les fichiers `.deca` destinés à ce mode doivent rester dans ce cadre, l'usage d'un mot-clé réservé au langage objet entraîne un rejet.

1.2 Sous-ensembles du langage, avec objet

Ce niveau étend le compilateur pour supporter les fonctionnalités orientées objet : définition et manipulation de classes, champs et méthodes.

L'utilisateur dispose alors d'un langage permettant de définir des hiérarchies de classes, d'encapsuler des données via des champs et des méthodes, de gérer la visibilité et d'effectuer des appels de méthodes. Cette extension nécessite des structures internes supplémentaires côté compilateur, notamment des environnements de classes et des tables de méthodes.

La vérification contextuelle du langage complet nécessite trois passes sur l'AST : vérification des noms de classes et de la hiérarchie ; vérification des signatures de méthodes et des déclarations de champs ; puis vérification des corps de méthodes, des blocs et des expressions.

Ces passes permettent de disposer des informations nécessaires au bon ordre : la hiérarchie et l'existence des classes doivent être établies avant le typage des champs et des signatures ; l'enregistrement des champs et méthodes est nécessaire avant l'analyse des corps. Enfin, le langage objet a un impact direct sur l'exécution la génération de code doit gérer une représentation mémoire des objets, l'initialisation des champs et un mécanisme d'appel de méthodes.

2 Commandes et options

La syntaxe officielle de la commande est : `decac [[-p | -v] [-n] [-r X] [-d]* [-P] [-w] <fichier deca>...] | [-b]`

Sans argument, `decac` affiche les options disponibles. `decac` peut être appelé avec un ou plusieurs fichiers sources `.deca`. Si un fichier apparaît plusieurs fois sur la ligne de commande, il n'est compilé qu'une seule fois. En l'absence de `-b`, `-p` (et éventuellement `-w`), une compilation réussie ne produit aucun affichage.

La compilation d'un ou plusieurs fichiers produit, pour chaque source valide, un fichier `.ass` placé dans le même répertoire. Si un fichier ne compile pas, aucun assembleur exploitable ne doit être produit pour ce fichier. Lorsque plusieurs fichiers sont fournis, l'utilisateur doit être attentif au fait que le compilateur peut mutualiser certains éléments internes et que l'option `-P` peut influencer l'ordre d'affichage de certaines traces (uniquement en mode debug), sans changer le résultat final généré.

2.1 Options disponibles (comportement attendu)

- **-b** : affiche une bannière indiquant le nom de l'équipe. Cette option est exclusive : elle ne s'utilise qu'avec **decac -b**.
- **-p** : s'arrête après la construction de l'AST et affiche la décompilation de cet arbre.
- **-v** : s'arrête après la vérification contextuelle, sans produire de fichier **.ass**.
- **-n** : supprime certains tests à l'exécution (ceux décrits dans la sémantique).
- **-r X** : limite les registres banalisés à $R0..R\{X-1\}$, avec $4 \leq X \leq 16$.
- **-d** : active des traces de debug ; l'option peut être répétée pour augmenter la verbosité.
- **-P** : si plusieurs fichiers sont donnés, compile en parallèle.
- **-w** : autorise l'affichage de messages d'avertissement.

L'option **-p** est principalement utile pour diagnostiquer des problèmes au niveau de l'analyse syntaxique et de la construction de l'arbre, car elle permet de vérifier que le compilateur comprend la structure du programme comme attendu. L'option **-v** sert à vérifier la cohérence sémantique sans générer de code, ce qui est utile lorsque l'utilisateur souhaite isoler un problème de typage, de déclaration ou de règles contextuelles. Les options **-r** et **-n** ont un impact direct sur la génération et l'exécution : **-r** modifie les contraintes de ressources en registres, et **-n** modifie le niveau de sécurité des vérifications dynamiques.

L'option **-d** est destinée au débogage interne du compilateur. Elle n'est pas censée être utilisée par un utilisateur final en usage standard, mais elle peut aider en cas de diagnostic difficile, notamment pendant le développement ou lorsqu'on cherche à comprendre à quelle étape la compilation s'interrompt. L'option **-w** autorise l'émission d'avertissements ; ces avertissements ne bloquent pas la compilation, mais ils signalent des situations potentiellement problématiques, par exemple des constructions qui ne sont pas erronées au sens strict mais qui peuvent conduire à des erreurs d'exécution.

Exécution du code généré (IMA)

Après compilation, le programme peut être exécuté avec **ima**. Par défaut, **ima** assemble et exécute directement, et propose aussi des options utiles.

Du point de vue de l'utilisateur, l'exécution avec **ima** correspond à la phase finale de validation du programme : le fichier **.ass** est interprété selon les règles de la machine abstraite. Les erreurs à l'exécution peuvent apparaître à ce moment, même si la compilation a réussi, notamment lorsqu'une condition dynamique erronée survient (division par zéro, débordement mémoire, déréférencement, etc.). Par défaut, le code généré inclut des blocs d'erreur permettant d'aboutir à un arrêt explicite. Lorsque l'option **-n** est activée, certains de ces contrôles sont supprimés, ce qui peut modifier la visibilité des erreurs : le programme peut alors se terminer de manière abrupte ou produire un comportement indéfini.

L'utilisateur doit également comprendre que la machine abstraite impose des contraintes de ressources : la pile et le tas ont une taille finie, et certaines situations peuvent conduire à un débordement. Le compilateur peut générer un test de débordement de pile au début des blocs de code, afin d'anticiper ces erreurs. Ces mécanismes contribuent à rendre l'exécution plus sûre et plus conforme aux attentes du projet.

3 Message d'erreurs

Tous les messages d'erreurs (lexicales, syntaxiques, contextuelles, et limitations du compilateur) doivent être au format :

```
nom_fichier.deca:ligne:colonne: description
```

Ce format est important pour l'évaluation automatique.

Pour l'utilisateur, ce format apporte deux bénéfices majeurs. Premièrement, il permet d'identifier précisément l'emplacement du problème, ce qui facilite une correction rapide. Deuxièmement, il est compatible avec des outils externes (scripts de tests, éditeurs, interfaces) qui peuvent exploiter cette information pour naviguer directement vers la ligne concernée. La description doit rester informelle mais suffisamment explicite pour distinguer les causes : erreur de syntaxe, identificateur inconnu, incompatibilité de types, règle de visibilité, etc.

Les erreurs doivent être émises dès qu'elles sont détectées, et la compilation doit s'arrêter sur une erreur bloquante afin d'éviter l'accumulation de messages secondaires. Dans certains cas, des avertissements peuvent être produits si l'option `-w` est activée. Un avertissement n'empêche pas la génération de code, mais il signale une situation que l'utilisateur devrait vérifier.

3.1 Catégories d'erreurs côté compilation

3.2 Erreurs de syntaxe et erreurs liées aux options

Les erreurs de syntaxe correspondent aux violations des règles de la grammaire Deca ou à une utilisation incorrecte des options de la ligne de commande. Ces erreurs sont détectées avant toute vérification contextuelle ou génération de code, et entraînent l'arrêt immédiat de la compilation.

Concernant les options de compilation, plusieurs erreurs peuvent être levées lorsque les arguments fournis à `decac` ne respectent pas les contraintes attendues. Les messages suivants peuvent être retournés :

- `EXCEPTION OPTIONS - -r` attend un nombre de registres
- Nombres de registres incorrectes
- Le nombre de registres doit être entre 4 et 16
- Option ou fichier invalide : " + arg
- L'option `-b` doit être utilisée seule.
- Les options `-p` et `-v` sont incompatibles.

Ces erreurs indiquent soit une incohérence dans la combinaison des options, soit une valeur incorrecte fournie à une option nécessitant un paramètre. Elles sont indépendantes du contenu du fichier `.deca` et relèvent exclusivement de l'analyse de la ligne de commande.

Du point de vue de l'utilisateur, ces diagnostics permettent de corriger immédiatement l'appel au compilateur, avant toute tentative d'analyse du programme source.

3.3 Erreurs contextuelles

Les erreurs contextuelles sont détectées lors des vérifications sémantiques du programme, après que la structure syntaxique a été validée. Elles correspondent aux violations des règles du langage Deca concernant les types, les environnements, l'héritage, les affectations et les expressions.

Ces erreurs sont organisées selon les différentes passes de l'analyse contextuelle.

Erreurs contextuelles générales (règles 0.x) Les erreurs suivantes correspondent à des opérations partielles ou à des conditions invalides sur les environnements globaux :

- Opération partielle : Identificateur non déclaré : " + name + " (règle 0.1)
- Opération partielle : Type non déclaré : " + name + " (règle 0.2)

Ces diagnostics apparaissent lorsque des identificateurs ou des types sont utilisés sans avoir été préalablement déclarés dans l'environnement courant.

Passé 1 – Environnement des types et des classes Lors de la première passe, le compilateur vérifie la cohérence des déclarations de classes et de l'héritage. Les erreurs suivantes peuvent être signalées :

- 1.2 opération partielle sur l'environnement des types : Nom de classe déjà déclaré
- 1.3 condition `env_types(super) = (class(),)` : L'identificateur étendu (`super-classe`) n'est pas une classe valide ou n'est pas déclaré
- Opération partielle `env_types(super)` : identificateur non déclaré
- Condition : identificateur de classe attendu
- Opération partielle classe ou type déjà déclaré (1.3)

Ces erreurs traduisent des incohérences dans la déclaration des classes, notamment des conflits de noms ou des références à des super-classes inexistantes ou invalides.

Passé 2 – Champs, méthodes et signatures La deuxième passe concerne la vérification des champs et des méthodes des classes, ainsi que leur compatibilité avec l'héritage. Les erreurs suivantes peuvent être levées :

- 2.3 Condition et opération partielle `env_types(super)` : erreur interne
- 2.3 Opération partielle : un nom de méthode redéclare un nom de champ
- 2.4 Opération partielle : nom d'attribut déjà déclaré
- 2.5 Condition `type != void` : Le type d'un champ ne peut pas être void
- 2.5 Conflit avec l'environnement de la super-classe (si défini, doit être un champ) : Un champ ne peut pas masquer une méthode héritée
- 2.6 Opération partielle (ajout d'une méthode) : Nom de méthode déjà déclaré dans la classe
- 2.7 Signature différente de la méthode héritée : La signature de la méthode redéfinie diffère de celle de la super-classe
- 2.7 Sous-typage du retour invalide (`subtype`) : Le type de retour n'est pas un sous-type de celui de la méthode héritée
- 2.9 Condition `type != void` : Le type d'un paramètre ne peut pas être void

- 2.9 Opération partielle (ajout d'un paramètre) : Nom de paramètre déjà utilisé

Ces erreurs concernent principalement les conflits de noms, les violations des règles d'héritage et les incohérences dans les signatures de méthodes.

Passe 3 – Corps des méthodes, expressions et instructions La troisième passe vérifie le contenu des blocs, des expressions et des instructions. Les diagnostics suivants peuvent être produits :

- 3.17 Condition type != void : Le type d'une variable ne peut pas être void (règle 3.17)
- 3.17 Opération partielle (ajout de variable) : Variable déjà déclarée dans ce bloc (règle 3.17)
- 3.24 Condition return != void : Impossible de retourner une valeur dans une méthode de type void
- 3.28 Condition assign_compatible : Type incompatible avec le type attendu (règle 3.28)
- 3.29 La condition doit être de type boolean : Condition non booléenne (règle 3.29)
- 3.31 Condition type = int ou float ou string : Type non supporté par print (règle 3.31)
- 3.33 Type_binary_op indéfini (types incompatibles)
- 3.37 L'opérande du not doit être de type booléen
- 3.39 Condition cast_compatible : Cast invalide
- 3.40 Type_instanceof_op indéfini : Opération instanceof invalide
- 3.42 New sur un type qui n'est pas une classe
- 3.43 Utilisation de 'this' interdite dans le programme principal
- 3.66 Accès illégal à un champ protégé

Ces erreurs reflètent des incohérences dans les expressions, les affectations, les conditions ou l'utilisation des constructions du langage.

3.4 Erreurs à l'exécution (Partie C)

Certaines erreurs ne peuvent être détectées qu'au moment de l'exécution du programme généré sur la machine abstraite IMA. Ces erreurs correspondent à des situations dynamiques invalides.

Les exceptions suivantes peuvent être levées :

- Division entier ou flottant par 0 : Erreur: division par zéro
- Stack overflow : Erreur: Débordement de la pile
- Le flottant est trop grand apres loperation : Erreur: Débordement flottant

Centralisation des erreurs à l'exécution (blocs d'erreur et addErrorBlock). Au niveau de la génération de code, le compilateur adopte une stratégie de centralisation des erreurs d'exécution : au lieu de dupliquer, à chaque point de détection, une séquence complète d'affichage et d'arrêt, il génère des blocs d'erreur standardisés regroupés en fin de programme assembleur. Chaque site de détection effectue alors un branchement vers

l'étiquette (*label*) correspondant au bloc d'erreur approprié. Cette organisation rend le code généré plus homogène et facilite le suivi du comportement à l'exécution.

Structure générale d'un bloc d'erreur. Un bloc d'erreur suit un schéma constant :

- une étiquette (*label*) servant de point d'entrée unique pour le branchement ;
- un préambule d'affichage fournissant un contexte (par exemple l'introduction **Erreur ligne** suivie de la valeur du registre R1, lorsque cette information est disponible) ;
- un message spécifique décrivant l'erreur concernée ;
- une terminaison standard (saut de ligne et arrêt avec un statut d'erreur).

Blocs d'erreur générés et situations déclenchantes. Les principaux blocs générés (ou prévus) et leurs déclencheurs sont les suivants :

- **division_zero** : déclenché lors d'une division entière, d'un modulo ou d'une division flottante par zéro ;
- **debordement_flottant** : activé via BOV après une opération arithmétique flottante lorsque le résultat dépasse la plage représentable ;
- **pile_pleine** : activé via BOV à la suite d'un TSTO placé au début de blocs exécutables (programme principal, méthode ou initialisation) ;
- **tas_plein** : déclenché via BOV suite à une allocation mémoire (NEW) échouée ;
- **dereferencement_null** : activé via BEQ après comparaison avec #null lors d'un accès à un champ (sélection) ou d'un appel de méthode ;
- **cast_error** : généré lors d'un transtypage de classe invalide (type dynamique incompatible avec le type cible) ;
- **io_error** : déclenché lors d'une lecture **readInt** ou **readFloat** lorsque l'entrée ne correspond pas au format attendu ;
- **absence_return** : bloc associé aux méthodes non-void, activé si l'exécution atteint la fin du corps d'une méthode sans rencontrer d'instruction **return**.

Gestion des registres et stratégie de sauvegarde (spilling). Le compilateur s'appuie sur un gestionnaire de registres (RegManager) pour allouer les registres temporaires, typiquement dans l'intervalle R2 à R15. Lorsque tous les registres disponibles sont occupés, une stratégie de sauvegarde/restauration est appliquée : un registre temporaire est sauvegardé sur la pile avant réutilisation (PUSH), puis restauré après (POP). Cette stratégie permet de préserver la correction du calcul dans les expressions complexes malgré un nombre limité de registres, tout en restant conforme aux conventions d'exécution de la machine cible.

Ces erreurs sont détectées par le code généré lorsque les vérifications à l'exécution sont activées. Elles entraînent un arrêt immédiat du programme.

Concernant la génération de code, certaines contraintes internes doivent être respectées. En particulier, les opérations d'affectation et les opérateurs logiques ne doivent jamais être traités par les mécanismes génériques de génération d'expressions. Les erreurs suivantes correspondent à des violations de ces contraintes :

- Ne jamais appeler **writeOperation** sur un **Assign**
- Ne jamais appeler **writeOperation** sur un **And**
- Ne jamais appeler **writeOperation** sur un **Or**

Les opérateurs `=`, `&&` et `||` doivent être redéfinis explicitement et ne peuvent pas être traités comme des opérations binaires classiques. Le mécanisme `AbstractBinaryOp` est utilisé pour les opérations arithmétiques et de comparaison (`+`, `-`, `*`, `/`, `<`, `>`, etc.).

Enfin, afin de garantir la cohérence du code généré, un gestionnaire de labels est utilisé pour assurer l'unicité des noms de labels. Cette séparation des préoccupations permet d'éviter les collisions de labels lors de la génération de code, en particulier dans les structures de contrôle et les expressions complexes.

4 Limitations

Les limitations décrites dans cette section correspondent aux cas identifiés et analysés au cours du développement et de la campagne de tests. Elles concernent principalement certains cas limites de la syntaxe, ainsi que des fonctionnalités volontairement non implémentées.

Il est toutefois possible que d'autres limitations existent et n'aient pas été mises en évidence, soit parce qu'elles relèvent de scénarios peu usuels du langage, soit parce qu'elles n'ont pas été couvertes par les jeux de tests utilisés. Ces cas pourraient apparaître lors de l'exécution de programmes exploitant des combinaisons complexes de constructions du langage ou des comportements dynamiques spécifiques.

- Profondeur de la pile : L'exécution est limitée par la taille de la pile allouée par l'interpréteur IMA (10 000 mots par défaut)
- Une récursion trop profonde provoquera une erreur *pile_pleine*
- Capacité du tas : De même, l'allocation d'un trop grand nombre d'objets via `new` peut saturer le tas et déclencher l'erreur *tas_plein*.
- Registres restreints : Si l'utilisateur utilise l'option `-r 4`, l'évaluation d'expressions extrêmement complexes pourrait être limitée par l'efficacité du mécanisme de "spilling" (sauvegarde sur pile).
- Le compilateur crashe pour les littéraux flottants trop grands .
- Bien que la base de tests couvre les cas d'utilisation standards et les erreurs sémantiques répertoriées, des comportements limites (edge cases) liés à des structures de classes extrêmement imbriquées ou des expressions dépassant la capacité de stockage temporaire de la machine IMA pourraient révéler des limitations propres à la gestion des ressources matérielles.

Lors de l'exécution de la commande `mvn compile test`, certains tests du dépôt de référence échouent. Ces échecs ne correspondent pas à des erreurs de fonctionnement du compilateur, mais à des limitations assumées de l'implémentation ou à des différences de périmètre par rapport aux fonctionnalités attendues par certains tests.

Tests syntaxiques : `float_underflow.deca` et `float_overflow.deca`. Les fichiers `float_underflow.deca` et `float_overflow.deca` sont classés comme invalides dans les tests syntaxiques. Toutefois, dans l'état actuel du compilateur, ces cas ne sont pas rejetés au niveau syntaxique ou contextuel.

En effet, la gestion fine des débordements et sous-dépassements flottants relève du comportement à l'exécution (Partie C) et non d'une vérification statique complète à la compilation. Le compilateur ne réalise pas d'analyse statique permettant de déterminer si

une constante flottante entraînera systématiquement un dépassement de capacité à l'exécution. Ces situations sont donc traitées dynamiquement via les blocs d'erreur d'exécution lorsque les vérifications sont activées.

Tests contextuels : répertoire `not_sorted`. Certains tests contextuels échouent car le répertoire `not_sorted` ne contient pas directement de fichiers `.deca` à analyser, mais uniquement des sous-répertoires (notamment des tests liés aux transtypes).

Le framework de tests s'attend à trouver des fichiers `.deca` à ce niveau, ce qui provoque une erreur lors de l'exécution des tests. Les fichiers présents dans les sous-répertoires, en particulier ceux liés aux casts, sont en revanche correctement traités par le compilateur et passent individuellement les tests attendus. Cette situation relève donc d'une organisation spécifique du jeu de tests plutôt que d'un défaut du compilateur.

Tests de génération de code : `exmathdoc.deca`. Le test `exmathdoc.deca` échoue avec un message indiquant une fonctionnalité « not yet implemented ». Ce comportement est attendu et cohérent avec les choix d'implémentation du projet.

Ce test repose sur l'extension `TRIGO`, qui n'a pas été implémentée dans le cadre de ce projet. À la place, l'équipe a choisi de développer une extension alternative et indépendante : une génération de code ciblant l'architecture ARM. Par conséquent, les fonctionnalités mathématiques avancées requises par `exmathdoc.deca` ne sont pas disponibles, et le compilateur signale explicitement l'absence d'implémentation.

4.1 Limitations du sous-ensemble Sans Objet

Le sous-ensemble sans objet vise le support des programmes sans classes. Les limitations ci-dessous correspondent à des restrictions attendues de ce mode :

- Les constructions orientées objet ne sont pas disponibles dans ce sous-ensemble : `class`, `extends`, `new`, `this`, `null`, `instanceof` et `return`, ainsi que la sélection `.` et les appels de méthodes sur objets.
- L'environnement de types est limité aux types de base (ex. `int`, `float`, `boolean`, `void`). Les types classes et `Object` n'ont de sens que dans le mode avec objet.

Pour l'utilisateur, ces limitations imposent d'écrire un programme centré sur un bloc principal et sur les constructions impératives de base. Toute tentative d'introduire des mécanismes objets est rejetée, soit dès la syntaxe, soit lors des vérifications contextuelles. En contrepartie, l'utilisateur bénéficie d'un cadre stable pour valider les aspects fondamentaux, et peut utiliser les options de compilation pour explorer le comportement du compilateur sous contraintes.

La séparation en sous-ensembles facilite aussi l'interprétation des erreurs : un message signalant l'usage de `class` ou `new` indique d'abord que le programme n'appartient pas au sous-ensemble courant, plutôt qu'un problème sémantique plus profond.

4.2 Limitations du langage Avec Objet

Le mode avec objet étend le compilateur à la manipulation de classes, champs, méthodes et héritage. Dans l'état actuel :

- Décompilation des classes incomplète : l'option `-p` n'est pas un outil fiable pour visualiser la partie `class` du programme (décompilation partielle ou générique). Pour inspecter le code objet, il est préférable de s'appuyer sur les tests de syntaxe et les diagnostics contextuels.
- Cas limites impliquant `null` : certains scénarios avec `null` (par exemple cast, assignations, compatibilités de types) peuvent être mal gérés, car les opérations internes sur le type `null` ne sont pas entièrement finalisées.
- Sensibilité aux ambiguïtés de grammaire : certaines combinaisons complexes de cast, sélection (`.`), appels chaînés et `instanceof` peuvent déclencher une erreur interne si l'analyseur ANTLR entre en mode *full-context*. Dans ce cas, l'utilisateur peut observer une exception interne au lieu d'un message `fichier:ligne:colonne`.

Dans un programme orienté objet, l'utilisateur s'attend à ce que l'analyse contextuelle reflète les règles d'héritage, de visibilité et de typage. Les limitations ci-dessus indiquent que, même si de nombreux programmes sont correctement traités, certains cas particuliers peuvent être instables, notamment lorsque plusieurs mécanismes du langage sont combinés dans une même expression.

5 Utilisation de l'extension

Dans l'état actuel du projet, et pour le rendu du lundi 19 janvier, **aucune extension fonctionnelle complète n'est livrée**. Le compilateur `decac` est donc utilisé, du point de vue de l'utilisateur, exclusivement avec les options standard décrites dans les sections précédentes, et la génération de code cible par défaut la machine abstraite IMA.

L'utilisateur compile ses fichiers `.deca` sans configuration spécifique liée à une extension. Les mécanismes `#include` restent disponibles pour inclure des fichiers de bibliothèque, mais l'utilisateur ne doit pas supposer la présence de fonctionnalités additionnelles non effectivement implémentées. L'utilisation de symboles définis uniquement comme des stubs peut conduire à un comportement non garanti à l'exécution.

Néanmoins, dans le cadre du projet de génie logiciel, une **extension de génération de code pour l'architecture ARM a été initiée**. Cette extension n'est pas livrée comme une fonctionnalité stable, mais elle fait l'objet d'une conception et d'un début d'implémentation. La présente section décrit donc le **mode d'utilisation prévu** de cette extension, tel qu'il est envisagé à terme.

5.1 Extension de génération de code pour architecture ARM

L'extension ARM vise à permettre la génération de code assembleur pour une architecture réelle, en particulier **ARM 32 bits sous Linux**, en alternative à la machine abstraite IMA. Elle ne modifie ni la syntaxe ni la sémantique du langage Deca : seules les étapes finales de génération de code sont concernées.

5.1.1 Commande et option associée

L'activation de l'extension est prévue via une option dédiée de la ligne de commande :

```
decac -arm programme.deca
```

En l'absence de cette option, le comportement du compilateur reste inchangé et la génération de code cible IMA. Dans l'état actuel du projet, cette option est documentée mais ne correspond pas à une fonctionnalité livrée dans la branche master du lundi 19 janvier 2026.

5.1.2 Code généré et exécution

Lorsque l'extension ARM est activée, la compilation est prévue pour produire un fichier `.ass` contenant du code assembleur ARM.

Le code généré n'est pas destiné à être exécuté avec l'outil `ima`. L'exécution repose sur la chaîne d'outils GNU ARM, en particulier :

- `arm-linux-gnueabi-hf-gcc` pour l'assemblage et le linkage ;
- `qemu-arm` pour l'exécution sur une machine hôte.

Ceci exécuté avec un script à la racine du projet à l'aide de : `./arm.sh fichier.ass`

Les opérations d'entrée/sortie et la terminaison du programme sont réalisées à l'aide des appels système Linux (*syscalls*).

5.1.3 Messages d'erreur

Les erreurs lexicales, syntaxiques et contextuelles sont détectées et signalées de la même manière que pour la compilation vers IMA.

Les erreurs à l'exécution dépendent du comportement du code ARM généré et des appels système utilisés. Elles peuvent donc différer des messages produits lors de l'exécution sur la machine abstraite IMA.

5.1.4 Limitations de l'extension

Dans son état actuel, l'extension ARM est limitée à un sous-ensemble restreint du langage Deca. Elle permet principalement l'exécution du bloc principal et l'affichage simple (`print`, `println`) et vise à englober la partie sans objet. Les fonctionnalités orientées objet (classes, méthodes, héritage, allocation dynamique) ne sont pas prises en charge.

L'extension ARM ne modifie pas les phases d'analyse du compilateur. Un programme accepté pour la compilation ARM respecte donc les mêmes règles de validité syntaxique et contextuelle que pour la compilation vers IMA, mais certaines erreurs dynamiques peuvent se manifester différemment à l'exécution.

5.1.5 Perspectives

Cette extension constitue une base de travail illustrant la faisabilité d'un compilateur multi-cibles. Elle ouvre la voie à une abstraction plus générale des backends, permettant à terme de supporter plusieurs architectures cibles sans modifier les phases d'analyse du compilateur.